

Assetleum 통합 테스트 결과서

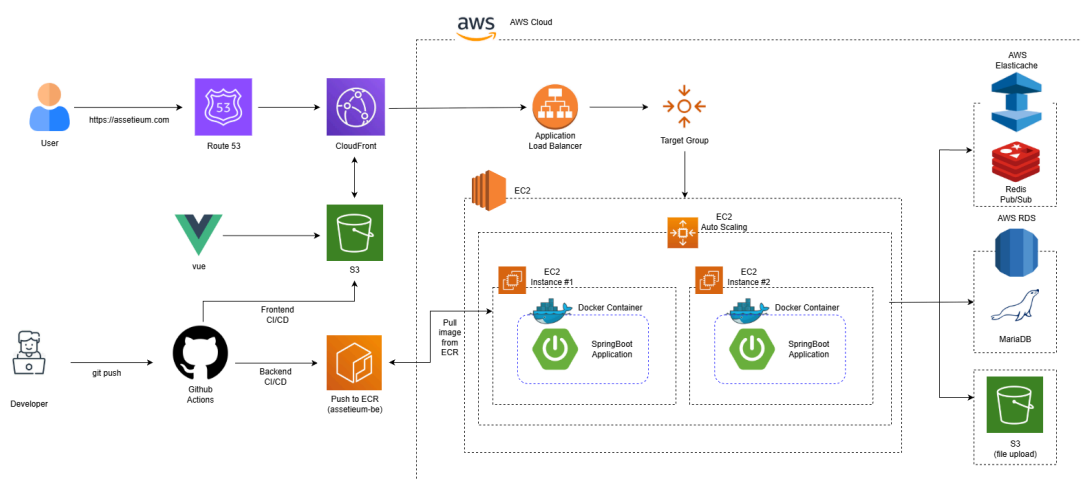
1. 주요 검증 대상

- 프론트엔드 및 백엔드 접속
- HTTPS 적용 상태
- ALB 및 Target Group 상태
- Auto Scaling 및 EC2 실행 상태
- RDS 데이터 저장 및 영속성
- ElastiCache Redis 연결 및 데이터 공유
- S3 파일 업로드 및 다운로드
- Security Group 접근 제한
- 실제 사용자 업무 시나리오

2. 기본 정보

항목	내용
프로젝트명	Assetleum
AWS 리전	ap-northeast-2 (Seoul)
프론트엔드 URL	https://assetieum.com
백엔드 URL	https://api.assetieum.com/api/v1
Health Check URL	https://api.assetieum.com/api/v1/health

3. 인프라 구성



4. 테스트 항목

ID	테스트 항목
T-01	프론트엔드 및 HTTPS 접속
T-02	백엔드 Health Check
T-03	ALB 및 Target Group 상태
T-04	Auto Scaling 및 EC2 실행 상태
T-05	RDS 연결 및 데이터 연속성
T-06	ElastiCache Redis 연결 및 공유
T-07	S3 파일 업로드 및 다운로드
T-08	Security Group 및 사용자 시나리오

5. 상세 테스트 결과

T-01. 프론트엔드 및 HTTPS 접속

테스트 목적

CloudFront와 S3를 통해 제공되는 프론트엔드 서비스가 HTTPS 환경에서 정상적으로 접속되는지 확인한다.

테스트 방법

1. 브라우저에서 프론트엔드 도메인에 접속한다.
2. 로그인 화면 또는 메인 화면이 정상적으로 표시되는지 확인한다.
3. 주소창에서 HTTPS 인증서 경고가 없는지 확인한다.
4. 페이지를 새로고침한다.
5. 새로고침 후에도 화면이 정상적으로 표시되는지 확인한다.

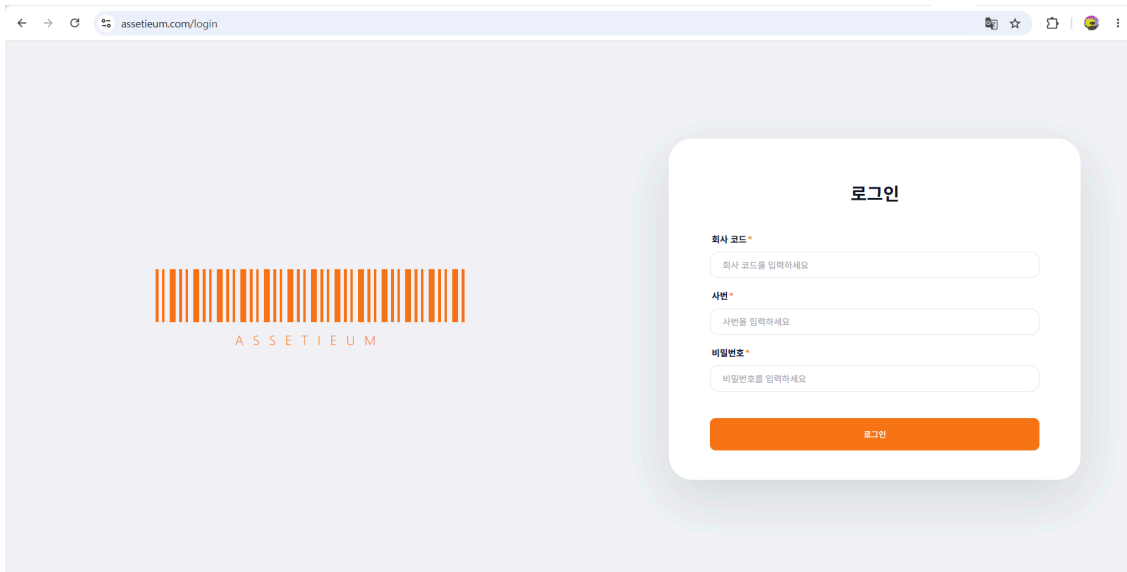
합격 기준

- 서비스 도메인 접속 성공
- HTTPS 인증서 오류 없음
- 화면과 정적 리소스 정상 로딩
- 새로고침 후 403,404 또는 흰 화면이 발생하지 않음

테스트 결과

확인 항목	결과
프론트엔드 접속	정상 접속 확인
HTTPS 적용	HTTPS 정상 적용 및 인증서 정상 동작 확인
정적 리소스 로딩	CSS, JavaScript, 이미지 정상 로드 확인
새로고침	새로고침 시 정상 동작 (404 오류 없음)
최종 판정	성공(PASS)

증빙 자료



T-02. 백엔드 Health Check

테스트 목적

ALB가 백엔드 애플리케이션의 정상 상태를 판단할 수 있도록 Health Check API가 정상적으로 동작하는지 확인한다.

실행 명령

```
[EC2 서버]
curl http://localhost:8080/api/v1/health

[도메인]
https://api.assetium.com/api/v1/health
```

합격 기준

- 인증 토큰 없이 호출 가능
- HTTP 상태 코드 200 반환
- 응답 데이터에 UP 표시
- 반복 호출 시 5xx 오류가 발생하지 않음

확인 결과

HTTP/2 200

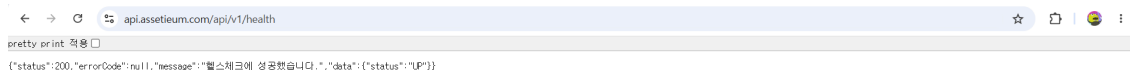
```
{
  "status": 200,
  "errorCode": null,
  "message": "헬스체크에 성공했습니다.",
  "data": {
    "status": "UP"
  }
}
```

```
}  
}
```

테스트 결과

확인 항목	결과
인증 없이 호출	성공
HTTP 상태 코드	200
서비스 상태	UP
최종 판정	PASS

증빙 자료



The screenshot shows a web browser window with the address bar displaying `api.assetleum.com/api/v1/health`. Below the address bar, there is a "pretty print" button and a JSON response: `{ "status": 200, "errorCode": null, "message": "웹스케크에 성공했습니다.", "data": { "status": "UP" } }`.

T-03. ALB 및 Target Group 상태

테스트 목적

외부 요청이 ALB 를 통해 정상적으로 수신되고 Target Group에 등록된 EC2 인스턴스로 전달되는지 확인한다.

확인 경로

AWS Console

- EC2
- Load Balancers
- Assetleum ALB

AWS Console

- EC2
- Target Groups
- Assetleum Target Group
- Targets

확인 항목

- ALB 상태가 Active
- HTTPS 443 Listener 존재
- Listener 요청이 Target Group으로 Forward
- Target Group Health Check 경로가 /api/v1/health
- Target Group에 등록된 대상이 Healthy

합격 기준

- ALB 상태가 Active
- HTTPS Listener가 정상적으로 설정됨
- Target Group 연결이 정상임
- 등록된 대상이 모두 Healthy
- 외부 Health Check 요청이 HTTP 200을 반환함

테스트 결과

확인 항목	결과
ALB 상태	정상
HTTPS 443 Listener	정상
Target Group 연결	정상
Health Check 경로	정상
Target Health	Healthy
최종 판정	성공(PASS)

증빙 자료

ALB의 HTTPS(443) 및 HTTP(80) Listener 설정과 Target Group 연결 상태 확인

리스너 및 규칙 (2) 정보

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against a rule. If no rule matches, the default action is applied.

Q 리스너 필터링

☐

프로토콜: 포트

▼

기본 작업

▼

규칙

▼

☐

[HTTPS:443](#)

• 대상 그룹으로 전달

[assetieum-tg](#) ↗ 1 (100%)

대상 그룹 고정성: 끄

☐

[HTTP:80](#)

• 리디렉션 대상 [HTTPS://#{host}:443/#{path}?#{query}](#)

상태 코드: HTTP_301

[1개 규칙](#)

[1개 규칙](#)

이름	포트	영역	상태 확인
-	8080	ap-northeast-...	✓ Healthy
	8080	ap-northeast-...	✓ Healthy

T-04. Auto Scaling 및 EC2 실행 상태

테스트 목적

Auto Scaling Group이 설정된 용량에 따라 EC2 인스턴스를 유지하고 각 인스턴스에서 백엔드 애플리케이션이 정상 실행되는지 확인한다.

확인 경로

```
AWS Console
└ EC2
  └ Auto Scaling Groups
    └ AssetIeum Auto Scaling Group
```

확인 항목

- Min Capacity 설정
- Desired Capacity 설정

- Max Capacity 설정
- 현재 인스턴스 수가 Desired Capacity와 일치
- 인스턴스 Lifecycle 상태가 InService
- 인스턴스 Health 상태가 Healthy
- 올바른 Launch Template 사용

EC2 확인 명령

```
docker ps
```

```
docker logs --tail 100 assetieum-be
```

합격 기준

- Auto Scaling Group이 Desired Capacity를 유지함
- EC2 인스턴스가 InService 상태임
- EC2 Health 상태가 Healthy임
- Docker 컨테이너가 Up 상태임
- Spring Boot 애플리케이션이 정상적으로 시작됨
- 반복 재시작 또는 치명적인 오류 로그가 없음

선택 테스트: Scale-out

1. EC2에 CPU 부하를 발생시킨다.
2. Auto Scaling Group이 새 EC2 인스턴스를 생성하는지 확인한다.
3. 새 인스턴스가 InService 상태로 변경되는지 확인한다.
4. Target Group에 등록되어 Healthy 상태가 되는지 확인한다.
5. CPU 부하 종료 후 Scale-in 테스트를 진행한다.

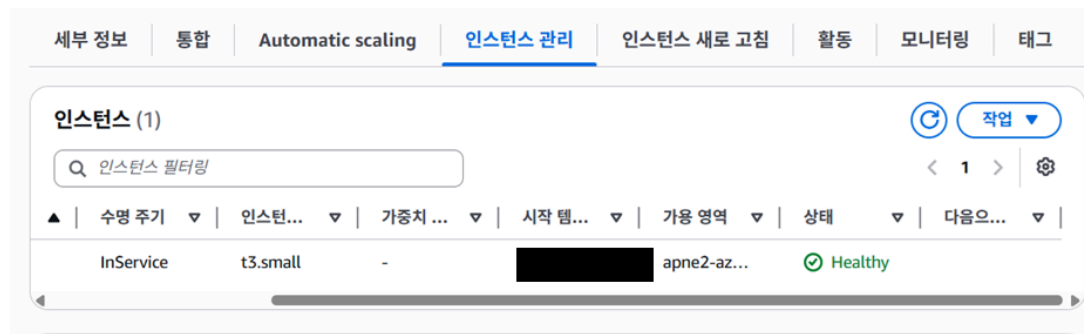
테스트 결과

확인 항목	결과
Min / Desired / Max	설정값 정상 확인 (예: 1 / 1 / 3)
현재 인스턴스 수	1대 정상 운영
EC2 Lifecycle 상태	InService 확인
EC2 Health 상태	Healthy 확인
Docker 컨테이너 상태	컨테이너 정상 실행(Up)
Spring Boot 실행 상태	Spring Boot 정상 실행 확인
Scale-out 테스트	수행
Scale-out 결과	새 EC2 자동 생성 및 Target Group Healthy 확인

확인 항목	결과
최종 판정	성공(PASS)

증빙 자료

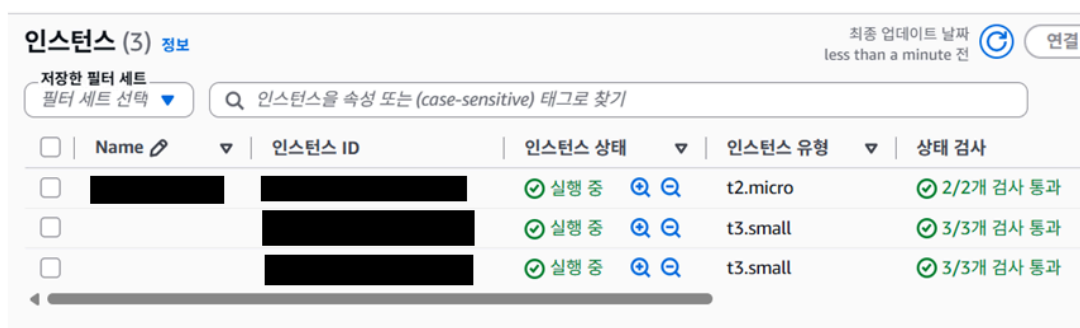
Auto Scaling Group InService 및 Healthy 화면



Scale-Out

(화면에는 기존 EC2 인스턴스도 함께 표시되지만 Auto Scaling 테스트는 ASG에 포함된 인스턴스를 대상으로 수행하였다.)

CPU 부하 발생 후 CloudWatch Alarm에 의해 Auto Scaling이 동작하여 EC2 인스턴스가 1대에서 2대로 증가하였다.



Scale-In

CPU 부하 종료 후 CloudWatch Alarm에 의해 Auto Scaling이 동작하여 EC2 인스턴스가 2대에서 1대로 감소하였다.

인스턴스 (1/3) 정보 최종 업데이트 날짜
less than a minute 전 [연결](#)

저장한 필터 세트
 필터 세트 선택

☐	Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사
☐	[REDACTED]	[REDACTED]	실행 중	t2.micro	2/2개 검사 통과
☐	[REDACTED]	[REDACTED]	종료됨	t3.small	-
☒	[REDACTED]	[REDACTED]	실행 중	t3.small	3/3개 검사 통과

Min, Desired, Max 값

원하는 용량

그룹 크기를 지정하세요.

크기 조정 한도

원하는 용량을 늘리거나 줄일 수 있는 양의 한도를 설정합니다.

원하는 최소 용량

원하는 용량보다 작거나 같음

원하는 최대 용량

원하는 용량보다 크거나 같음

EC2 Docker 및 Spring Boot 실행 화면

```

:~$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
o	Up 24 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	/assetieum-be:latest "java -jar app.jar" 24 hours ag

T-05. RDS 연결 및 데이터 영속성

테스트 목적

백엔드 애플리케이션이 RDS MySQL에 정상적으로 연결되고, EC2 또는 Docker 컨테이너가 재시작되어도 데이터가 유지되는지 확인한다.

확인 경로

AWS Console
→ RDS
→ Databases
→ Assetleum DB

테스트 절차

1. 서비스에서 테스트용 티켓 또는 데이터를 생성한다.

2. 생성된 티켓 번호 또는 데이터 식별값을 기록한다.
3. 백엔드 Docker 컨테이너를 재시작한다.
4. 서비스에 다시 접속한다.
5. 생성한 데이터가 정상적으로 조회되는지 확인한다.
6. 백엔드 로그에서 DB 연결 오류가 없는지 확인한다.

컨테이너 재시작 예시

docker restart assetieum-be

합격 기준

- RDS 상태가 Available
- 애플리케이션에서 데이터 생성 성공
- 생성한 데이터 조회 성공
- 컨테이너 재시작 후에도 데이터 유지
- DB Connection Timeout 오류 없음
- Too many connections 오류 없음

테스트 결과

확인 항목	결과
RDS 상태	Available 확인
데이터 생성	테스트 데이터 생성 성공
생성 데이터 조회	생성한 데이터 정상 조회
컨테이너 재시작	(docker restart assetieum-be) 정상 수행
재시작 후 데이터 조회	기존 데이터 정상 조회(데이터 유지 확인)
DB 연결 오류	오류 없음 (Connection Timeout 및 Too many connections 없음)
최종 판정	성공 (PASS)

증빙 자료

RDS 상태



재시작 후 화면

```

ubuntu@assetieum-be:~$ docker restart assetieum-be
ubuntu@assetieum-be:~$ docker ps
CONTAINER ID   IMAGE      PORTS          NAMES                COMMAND                CREATED
STATUS        PORTS      NAMES          COMMAND              CREATED
0 Up 7 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp latest "java -jar app.jar" 24 hours ag

```

자산이름

대시보드

유형자산

무형자산

서비스 데스크

나의 요청

전수조사

설정

서비스데스크 > 나의 요청

나의 요청

20개씩 보기

총 2건 중 1-2건

전체 상태

전체 유형

티켓 번호 또는 품목명 검색

조회하기

티켓 번호	티켓 유형	품목명	요청일시	진행 상태
TKT-20260629-00001	직접 구매 요청	MacBook Pro 14 M5	2026-06-29 01:22	요청됨
TKT-20260626-00001	직접 구매 요청	삼성 갤럭시 이온2	2026-06-26 05:26	처리 중

항사원

서원

로그아웃

T-06. ElastiCache Redis 연결 및 공유

테스트 목적

백엔드 애플리케이션이 ElastiCache Redis에 정상적으로 연결되고 여러 EC2 인스턴스가 동일한 Redis 데이터를 공유하는지 확인한다.

확인 경로

```

AWS Console
- ElastiCache
-> Redis OSS
t AssetIeum Cache

```

Redis 연결 테스트

```
redis-cli -h <ELASTICACHE_ENDPOINT> -p 6379 ping
```

정상 결과:

PONG

다중 인스턴스 공유 테스트

EC 2 A 에서 테스트 값을 저장한다.

```
redis-cli -h <ELASTICACHE_ENDPOINT> -p 6379 SET autoscaling: test "from-a" EX 300
```

EC 2 B 에서 동일한 값을 조회한다.

```
redis-cli -h <ELASTICACHE_ENDPOINT> -p 6379 GET autoscaling:test
```

정상 결과:

"from-a"

합격 기준

- ElastiCache 상태가 Available
- Redis PING 결과가 PONG
- EC2 A에서 저장한 값을 EC2 B에서 조회할 수 있음
- 운영 EC2에 별도 로컬 Redis 컨테이너가 실행되고 있지 않음

테스트 결과

확인 항목	결과
ElastiCache 상태	Available 확인
Redis PING	PONG 응답 확인

확인 항목	결과
EC 2 A 데이터 저장	SET 명령 성공(OK)
EC 2 B 데이터 조회	GET 결과 "from-a" 확인
EC 2 간 데이터 공유	정상 확인
로컬 Redis 컨테이너	없음
최종 판정	성공 (PASS)

증빙 자료

ElastiCache 및 Redis 공유 테스트 화면.

▼ 클러스터 세부 정보			
클러스터 이름 [REDACTED]	설명 -	노드 유형 cache.t3.micro	상태 🟢 Available
엔진 🔗 Redis	엔진 버전 7.1.0	글로벌 데이터 스토어 -	글로벌 데이터 스토어 역할 -
업데이트 상태 최신 상태	클러스터 모드 비활성화됨	샤드 1	노드 수 1
다중 AZ 비활성화됨	자동 장애 조치 비활성화됨	전송 중 암호화 활성화됨	저장 중 암호화 활성화됨

```
ubuntu@ ~$ redis-cli -h -p 6379 ping
PONG
ubuntu@ ~$
```

EC2 A에서 ElastiCache Redis에 테스트 데이터를 저장 후 EC2 B에서 같은 Redis 데이터를 조회

```
ubuntu@ ~$ redis-cli --tls -h -p 6379 SET autoscaling:test "from-ec2-a" EX 360
OK
ubuntu@ ~$ redis-cli --tls -h -p 6379 GET autoscaling:test
"from-ec2-a"
```

T-07. S3 파일 업로드 및 다운로드

테스트 목적

Assetleum 서비스에서 업로드한 파일이 EC2 로컬 디스크가 아닌 S3에 저장되고, 서버가 변경되거나 재시작되어도 정상적으로 다운로드되는지 확인한다.

테스트 절차

1. 서비스에서 테스트용 증빙 파일을 업로드한다.
2. 업로드 성공 응답을 확인한다.
3. AWS S3 버킷에서 업로드된 객체를 확인한다.
4. 객체의 Key, Size, Last modified, Content-Type을 확인한다.
5. 서비스에서 해당 파일을 다운로드한다.
6. 백엔드 컨테이너를 재시작하거나 다른 EC2인스턴스를 통해 다시 다운로드한다.
7. EC2 로컬 저장 경로에 파일이 남아 있지 않은지 확인한다.

EC2 로컬 파일 확인 명령

```
sudo find /uploads /opt/assetleum/files -type f 2>/dev/null
```

합격 기준

- 서비스에서 파일 업로드 성공
- S3 버킷에 객체 생성
- 파일 크기와 Content-Type 정상
- 서비스에서 파일 다운로드 성공
- 서버 재시작 후에도 다운로드 가능
- 다른 EC2 인스턴스에서도 다운로드 가능
- EC2 로컬 경로에 최종 파일이 남지 않음

테스트 결과

확인 항목	결과
서비스 파일 업로드	업로드 성공
S3 객체 생성	객체 생성 확인
객체 Key	정상 생성 확인
파일 Size	원본 파일과 동일 확인
Content-Type	정상 확인
서비스 파일 다운로드	다운로드 성공
서버 변경 후 다운로드	정상 다운로드 확인
EC2 로컬 파일 잔존	없음
최종 판정	성공 (PASS)

증빙 자료

파일 업로드 성공 화면

직접구매 상세 정보

실제 결제 금액

1,500,000원

구매처

홍플러스 서수원점

구매 일시

2026-06-26 05:26

증빙 파일명

-

증빙 업로드 일시

-

증빙 파일 목록

KakaoTalk_20260331_114851292.png

53 KB · 2026-06-30 01:22

S3 객체 상세 화면

ticket/

S3 URI 복사

객체

속성

객체 (1)

S3 URI 복사

URL 복사

다운로드

열기

삭제

작업

폴더 만들기

업로드

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. 다른 사용자가 객체에 액세스할 수 있게 하려면 명시적으로 권한을 부여해야 합니다. [자세히 알아보기](#)

점두사로 객체 찾기

< 1 >

이름

유형

마지막 수정

크기

스토리지 클래스

26f9705d-1d52-4972-92d3-666674a8952c.png

png

2026. 6. 30. am 10:22:37 AM KST

52.7KB

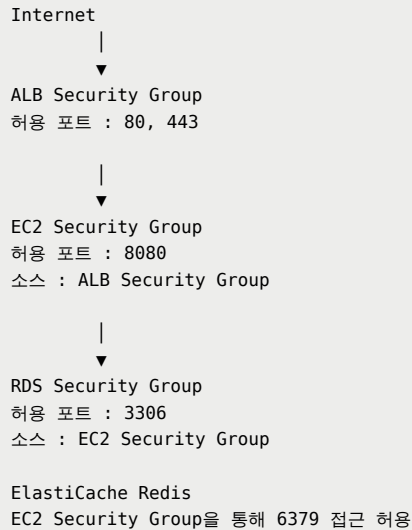
Standard

T-08. Security Group 및 최종 사용자 시나리오

테스트 목적

AWS 리소스 간 접근 경로가 필요한 범위로 제한되어 있는지 확인하고 실제 사용자가 수행하는 주요 업무 기능이 정상 적으로 동작하는지 검증한다.

Security Group 구성



Security Group 합격 기준

- 외부 인터넷에서는 ALB의 80,443 포트만 접근 가능
- EC2 애플리케이션 포트는 ALB Security Group에서만 접근 가능
- RDS 3306 포트는 EC2 Security Group에서만 접근 가능
- Redis 6379 포트는 EC2 Security Group에서만 접근 가능
- 불필요한 전체 포트 허용 규칙이 없음

최종 사용자 시나리오

1. 프론트엔드 도메인에 접속한다.
2. 테스트 사용자 계정으로 로그인한다.
3. 새로운 자산 요청 또는 테스트 요청을 생성한다.
4. 요청 목록과 상세 화면에서 생성 결과를 확인한다.
5. 다른 권한 사용자로 로그인한다.
6. 생성된 요청을 승인하거나 상태를 변경한다.
7. 요청자 계정으로 다시 확인한다.
8. 변경된 요청 상태가 정상적으로 반영되었는지 확인한다.
9. 알림이 정상적으로 수신되는지 확인한다.
10. 첨부 파일을 조회하거나 다운로드한다.
11. 로그아웃 후 다시 로그인한다.
12. 기존 요청과 변경된 상태가 유지되는지 확인한다.

사용자 시나리오 합격 기준

- 로그인 성공

- 요청 생성 성공
- 요청 목록 및 상세 조회 성공
- 권한에 따른 승인 또는 상태 변경 성공
- 요청자 화면에 변경 상태 반영
- 알림 수신 성공
- 첨부 파일 조회 또는 다운로드 성공
- 로그아웃 및 재로그인 성공
- 재로그인 후 기존 데이터 유지
- 테스트 과정에서 치명적인 5xx 오류가 발생하지 않음

테스트 결과

확인 항목	결과
ALB Security Group	HTTP(80), HTTPS(443) 인바운드 허용 확인
EC2 Security Group	ALB Security Group으로부터 8080 접근만 허용 확인

EC2 인바운드 규칙

IP 버전	유형	프로토콜	포트 범위
IPv4	HTTP	TCP	80
-	사용자 지정 TCP	TCP	8080
IPv4	HTTPS	TCP	443
-	사용자 지정 TCP	TCP	6379
IPv4	SSH	TCP	22

ALB 인바운드 규칙

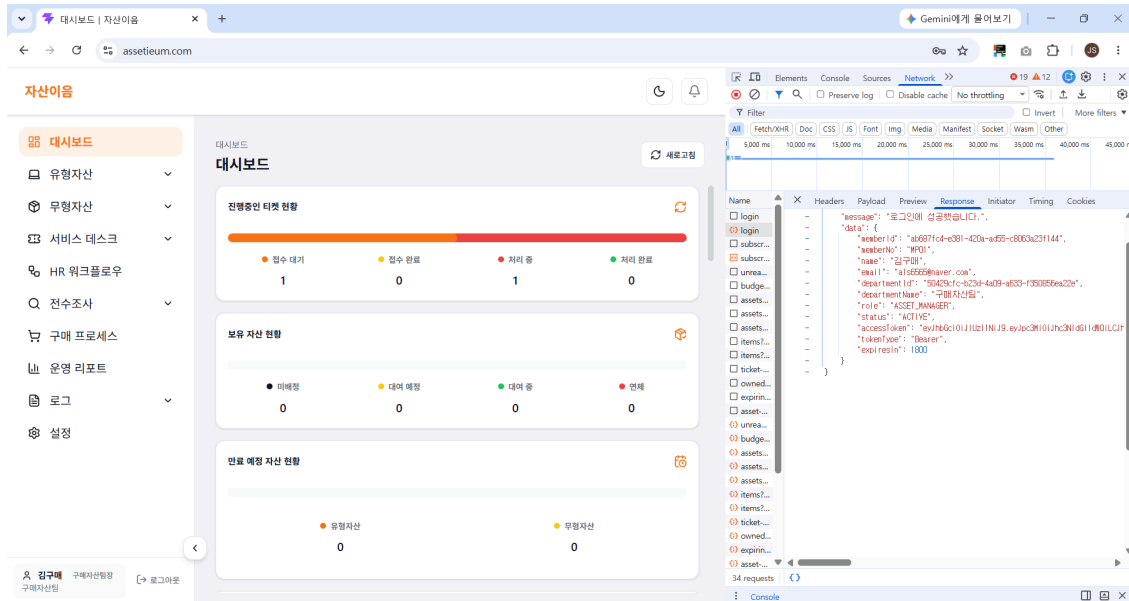
인바운드 규칙 (2)						
IP 버전	유형	프로토콜	포트 범위	소스		
IPv4	HTTPS	TCP	443	0.0.0.0/0		
IPv4	HTTP	TCP	80	0.0.0.0/0		

RDS 인바운드 규칙

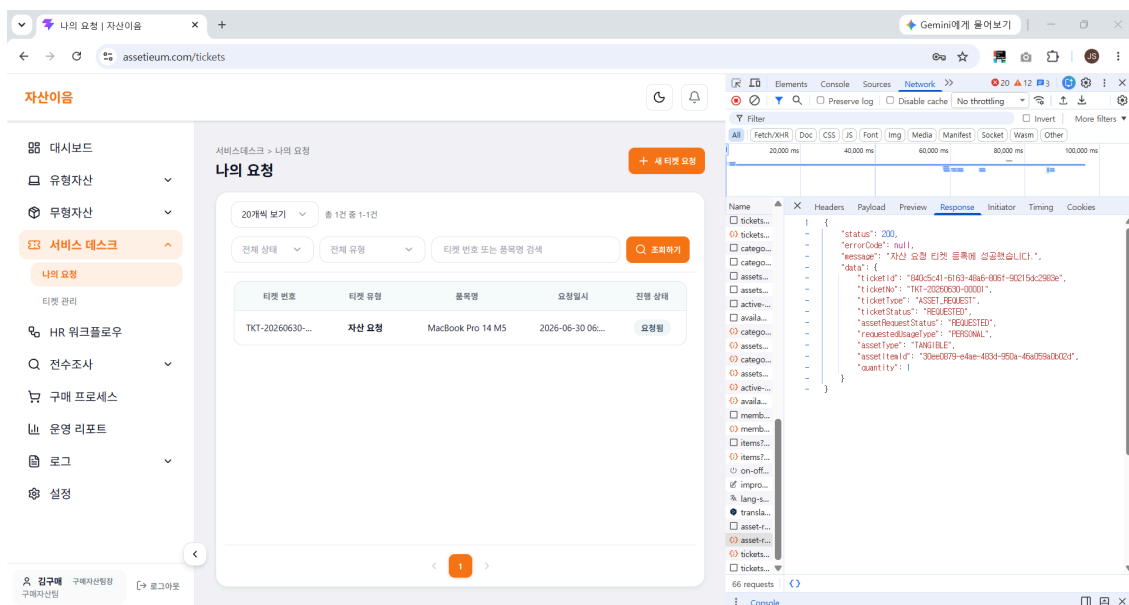
IP 버전	유형	프로토콜	포트 범위
IPv4	MYSQL/Aurora	TCP	3306
-	MYSQL/Aurora	TCP	3306

최종 사용자 업무 시나리오

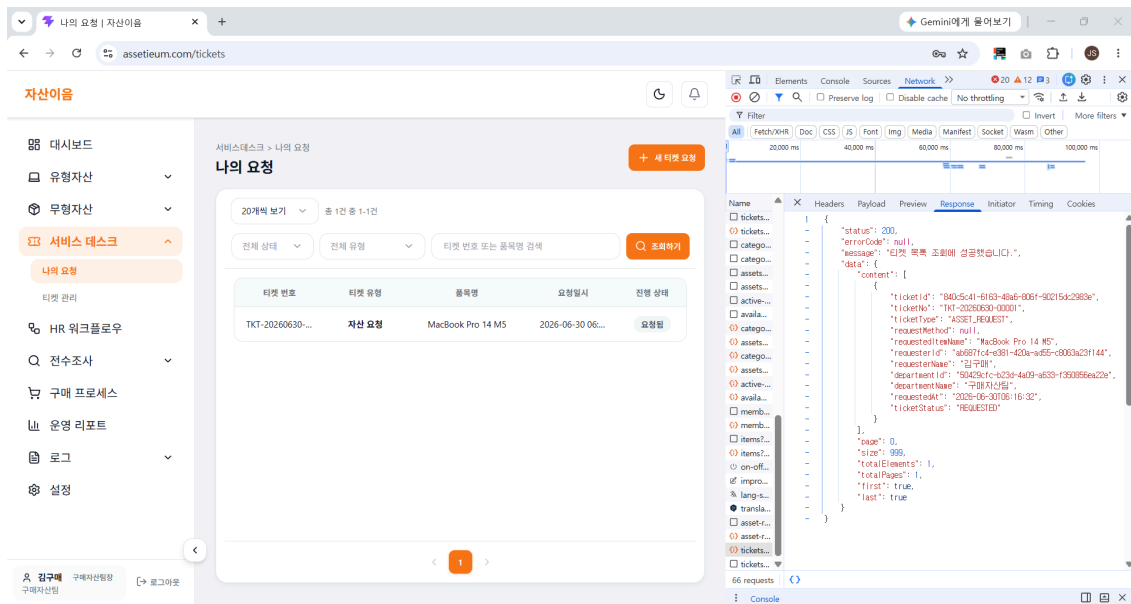
로그인



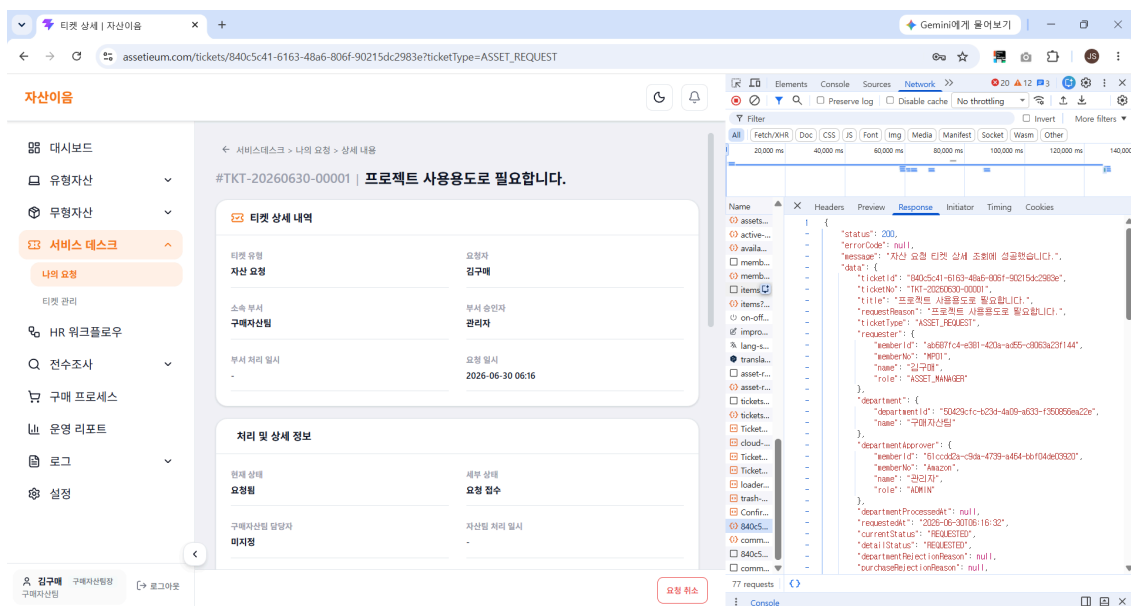
요청 생성



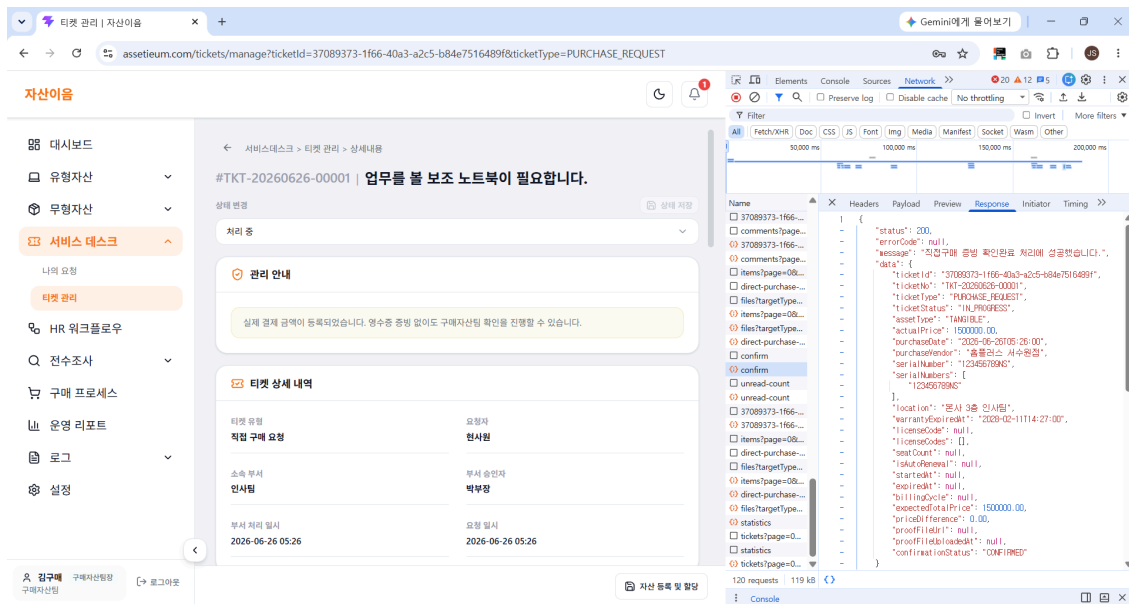
목록 및 상세 조회



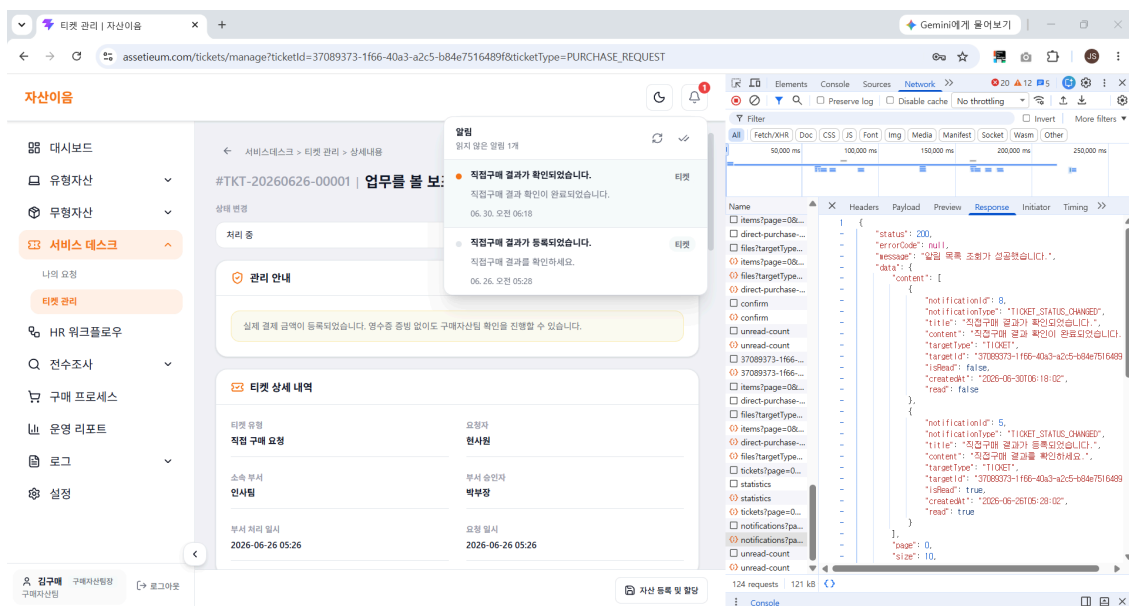
승인 또는 상태 변경



요청자 상태 반영



알림 수신



- 파일 조회 가능
- 재로그인 후 데이터 유지됨
- 5xx 오류 발생 여부 없음

종합 의견

Assetleum 서비스의 프론트엔드와 백엔드가 HTTPS 환경에서 정상적으로 제공되는지 확인하였다.

ALB와 Target Group의 연결 상태 및 Auto Scaling Group에 포함된 EC2 인스턴스의 실행 상태를 확인하였으며, EC2의 Docker 컨테이너에서 Spring Boot 애플리케이션이 정상적으로 실행되는지 검증하였다.

또한 RDS, ElastiCache, S3 연동 테스트를 통해 데이터와 파일이 특정 EC2 인스턴스의 로컬 환경에 종속되지 않고 저장되는지 확인하였다.

Security Group 설정을 통해 외부 접근 경로와 AWS 리소스 간 접근 범위가 제한되어 있는지 검토하였으며, 실제 사용 자 시나리오를 수행하여 로그인, 요청 생성, 조회, 승인, 알림 및 파일 다운로드 등의 주요 기능이 정상적으로 동작하는 지 검증하였다.

최종 테스트 결과, Assetleum 서비스는 AWS 인프라 환경에서 정상적으로 운영 가능한 상태임을 확인하였다.